

Lecture3

认识对象：封装数据为类



2024/9/26

Xiang Gao



北京航空航天大学
COLLEGE OF SOFTWARE
BEIHANG UNIVERSITY 软件学院

主要内容

- 类的设计原则
- Java程序的基本结构
- 类的定义
 - 成员变量的定义
 - 成员方法的定义
 - 构造函数定义
- 进一步理解对象
 - this关键字
 - 对象的生成、使用和清除
 - 进一步理解引用类型变量
 - 对象序列化
- 类的静态属性和静态方法
- 进一步理解封装

主要内容

- 类的设计原则
- Java程序的基本结构
- 类的定义
- 进一步理解对象
 - this关键字
 - 对象的生成、使用和清除
 - 进一步理解引用类型变量
 - 对象序列化
- 类的静态属性和静态方法
- 进一步理解封装
- 单例模式

- Java 每个类都默认地具有 `null`、`this`、`super` 三个域，所以在任何类中都可以不加说明就可以直接引用它们：
 1. `null` : 代表“空”，用在定义一个对象但尚未为其开辟内存空间时。
 2. `this`: 是常用的指代本类对象的关键字

this关键字

- this可以看做一个变量，他的值是当前对象的引用
- 在类的方法定义中使用this关键字代表使用该方法的对象的引用
- 使用this可以处理方法中成员变量和局部变量重名的问题

this关键字

为了区分两个name，用this代表使用该方法的对象mao的引用

```
class Cat{
    String name;
    String xb;
    int nl;
    public void set( String name,String xb,int nl){
        this.name=name;
        this.xb=xb;
        this.nl=nl;
    }
    public String toString(){
        return nl+"岁的"+name+xb+"睡着了";
    }
}

public class Test{
    public static void main(String[] args){
        Cat mao=new Cat();
        mao.set("花花","先生",2);
        System.out.println(mao);
    }
}
```

成员变量与方法
的参数同名

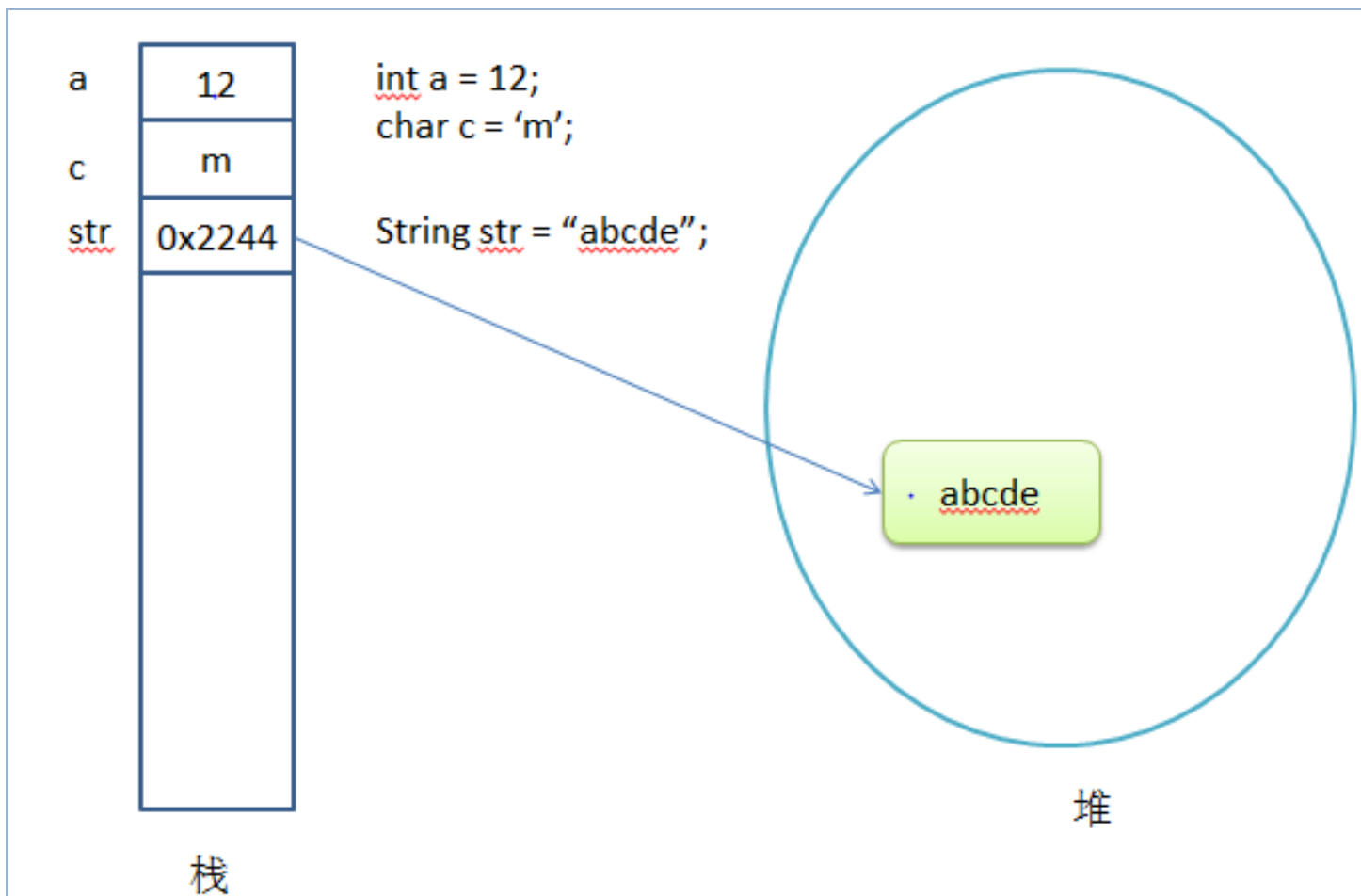
主要内容

- 类的设计原则
- Java程序的基本结构
- 类的定义
- 进一步理解对象
 - this关键字
 - 对象的生成、使用和清除
 - 进一步理解引用类型变量
 - 对象序列化
- 类的静态属性和静态方法
- 进一步理解封装
- 单例模式

创建对象

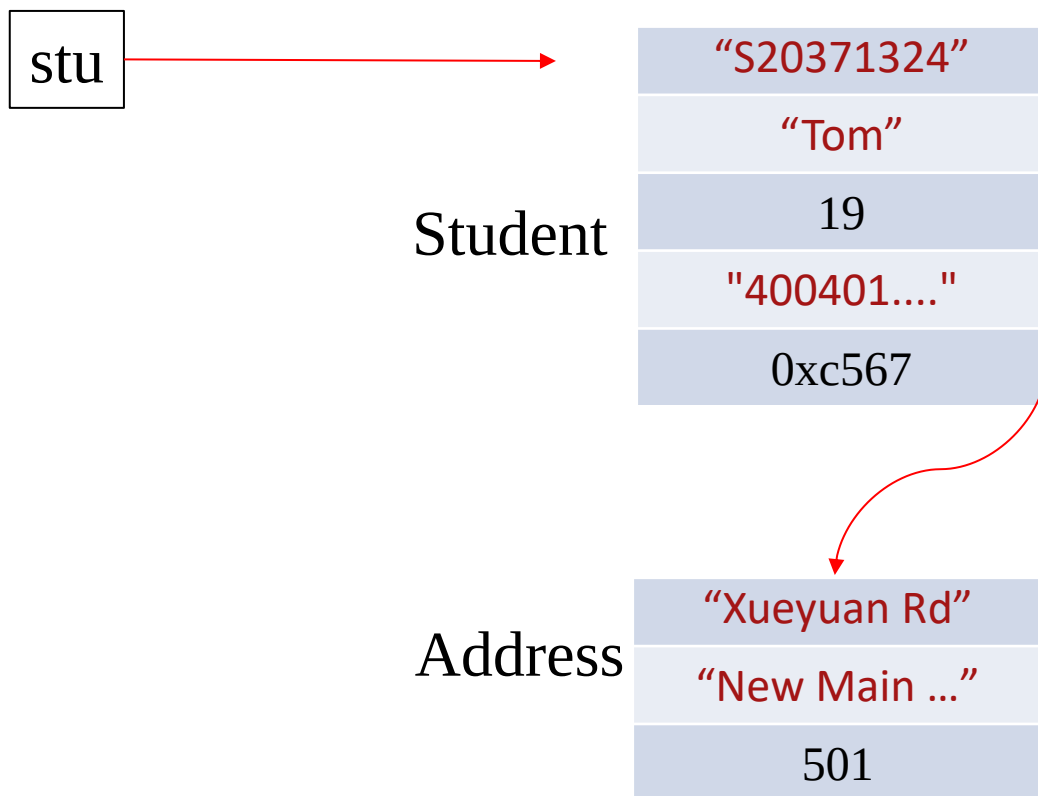
- 构造函数实例化类对象的格式
 - 类名 对象名=new 构造函数（实际参数）
 - New关键字的作用
 - ① 为对象分配内存空间。
 - ② 引起对象构造方法的调用。
 - ③ 为对象返回一个引用。

简单数据类型和复合数据类型内存分配



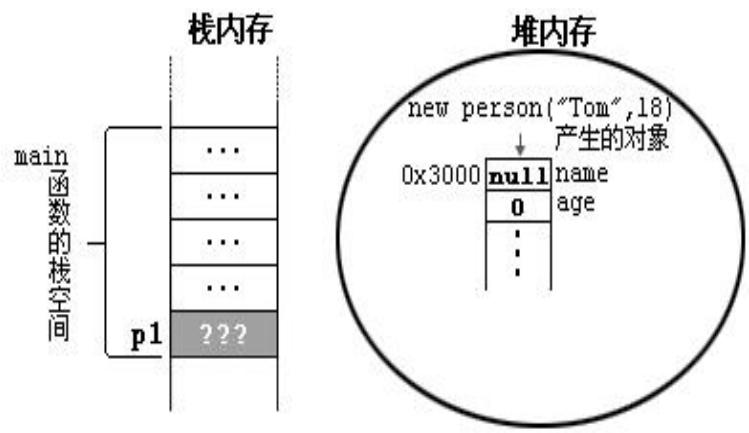
类是引用类型变量

```
Student stu = new Student("S20371324", "Tom", 19, "400401....",  
    new Address("Xueyuan Road", "New Main Build.", 501));
```

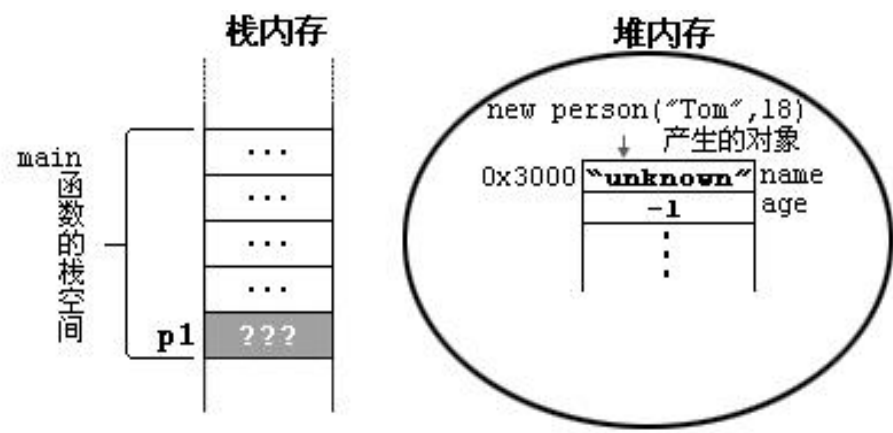


变量声明不为对象分配内存空间，只分配引用空间；对象的引用类似于指针，是32/64位地址空间，它的值指向堆中的数据结构。

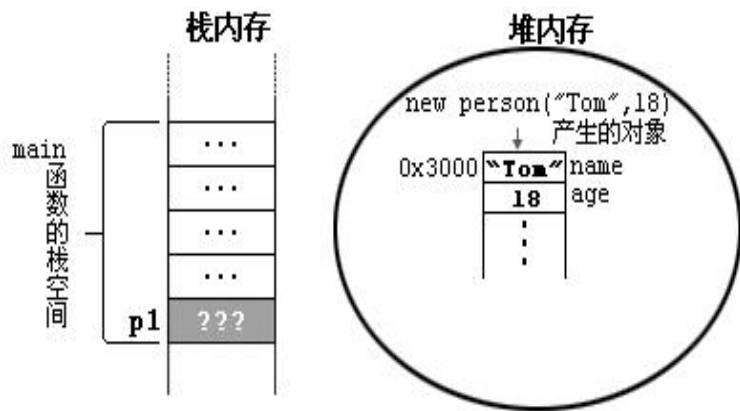
Person p1 = new Person("Tom",18) 的内存状态变化过程分析



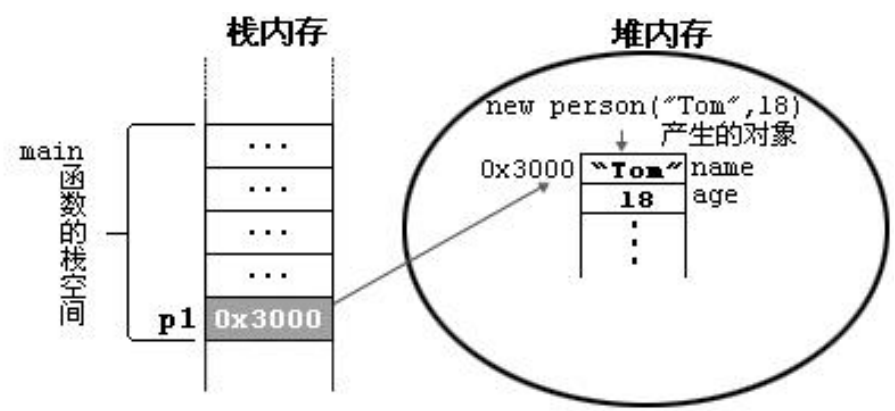
(2)



(1)



(3)

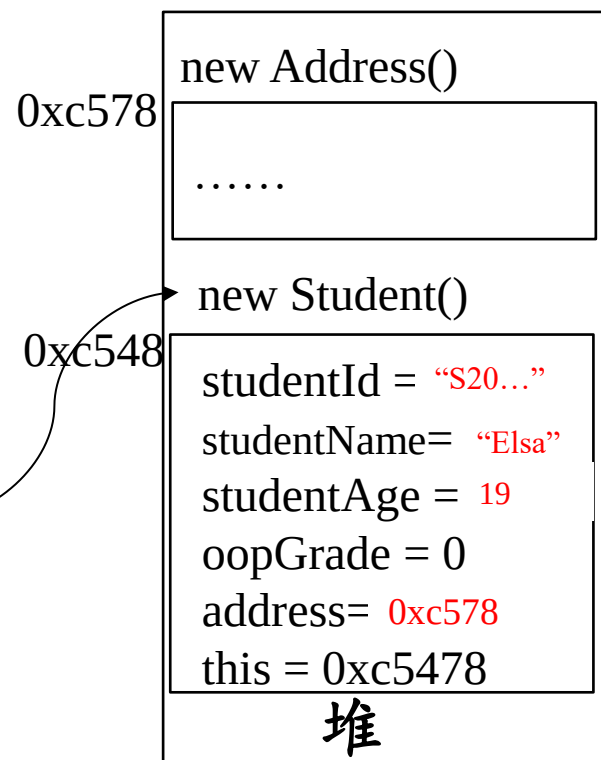
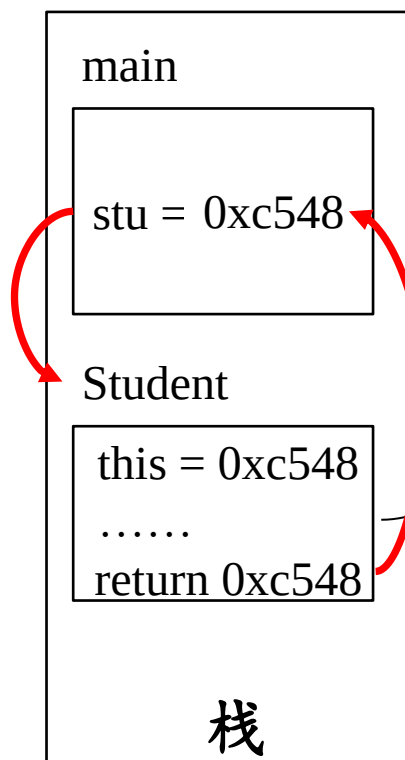


(4)

对象的内存图解

```
public static void main(String[] args){  
    Student stu = new Student(studentId: "S20371324", studentName: "Elsa",  
                               studentAge: 19, idNumber: "400401....",  
                               new Address(street: "Xueyuan Road", building: "New Main  
Building", unitNo: 501));  
    int age = stu.getAge();  
}
```

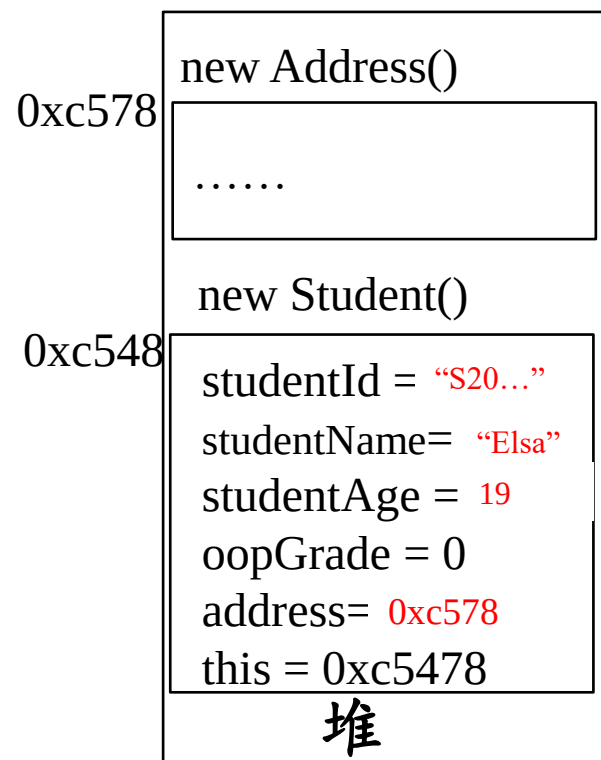
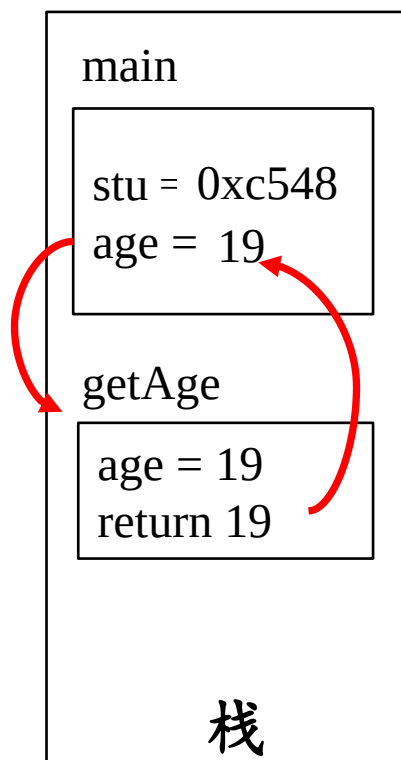
```
3 public class Student {  
4     protected String studentId = null;  
5     protected String studentName = null;  
6     protected int studentAge = 0;  
7     protected int oopGrade = 0;  
8     private final String idNumber;  
9     protected Address addr;  
10  
11     public Student(String studentId, String studentName, int studentAge, String idNumber, Address addr){  
12         this.studentId = studentId;  
13         this.studentName = studentName;  
14         this.addr = addr;  
15         this.idNumber = idNumber;  
16         this.setAge(studentAge);  
17     }  
18  
19  
20     public int getAge(){  
21         int age = 19;  
22         return age;  
23     }  
}
```



对象的内存图解

```
public static void main(String[] args){  
    Student stu = new Student(studentId: "S20371324", studentName: "Elsa",  
                               studentAge: 19, idNumber: "400401....",  
                               new Address(street: "Xueyuan Road", building: "New Main  
Building", unitNo: 501));  
    int age = stu.getAge();  
}
```

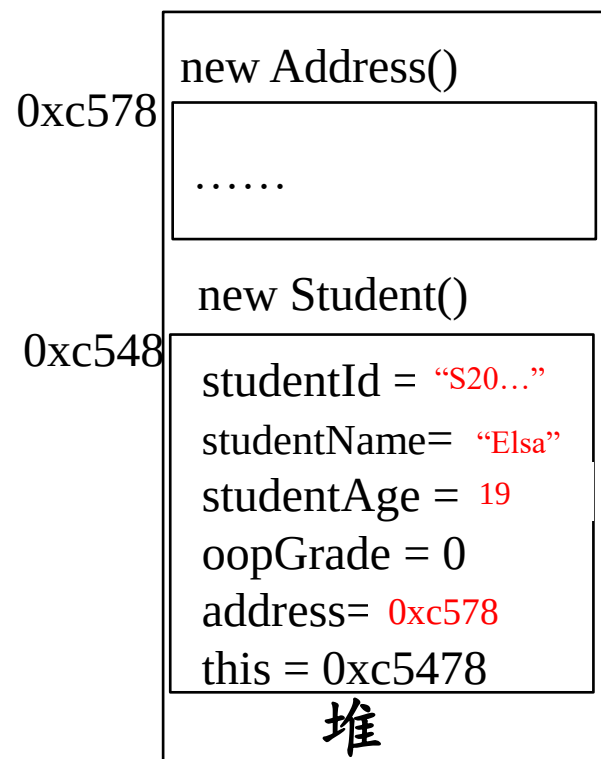
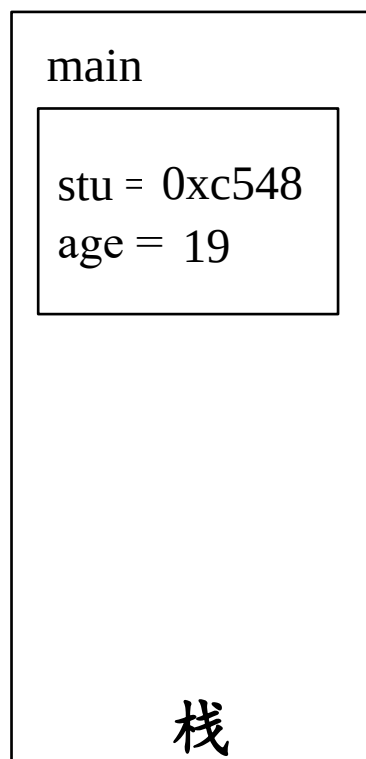
```
3 public class Student {  
4     protected String studentId = null;  
5     protected String studentName = null;  
6     protected int studentAge = 0;  
7     protected int oopGrade = 0;  
8     private final String idNumber;  
9     protected Address addr;  
10  
11     public Student(String studentId, String studentName, int studentAge, String idNumber, Address addr){  
12         this.studentId = studentId;  
13         this.studentName = studentName;  
14         this.addr = addr;  
15         this.idNumber = idNumber;  
16         this.setAge(studentAge);  
17     }  
18  
19  
20     public int getAge(){  
21         int age = 19;  
22         return age;  
23     }  
}
```



对象的内存图解

```
public static void main(String[] args){  
    Student stu = new Student(studentId: "S20371324", studentName: "Elsa",  
                               studentAge: 19, idNumber: "400401....",  
                               new Address(street: "Xueyuan Road", building: "New Main  
Building", unitNo: 501));  
    int age = stu.getAge();  
}
```

```
3 public class Student {  
4     protected String studentId = null;  
5     protected String studentName = null;  
6     protected int studentAge = 0;  
7     protected int oopGrade = 0;  
8     private final String idNumber;  
9     protected Address addr;  
10  
11     public Student(String studentId, String studentName, int studentAge, String idNumber, Address addr){  
12         this.studentId = studentId;  
13         this.studentName = studentName;  
14         this.addr = addr;  
15         this.idNumber = idNumber;  
16         this.setAge(studentAge);  
17     }  
18  
19  
20     public int getAge(){  
21         int age = 19;  
22         return age;  
23     }  
}
```



- 内存管理通常有两种方法：
 - ① 一种方法是由程序员在编写程序时显式地释放内存，例如C/C++。
 - ② 另一种方法是由语言的运行机制自动完成，例如Java。

JAVA内存管理 (垃圾自动回收机制)

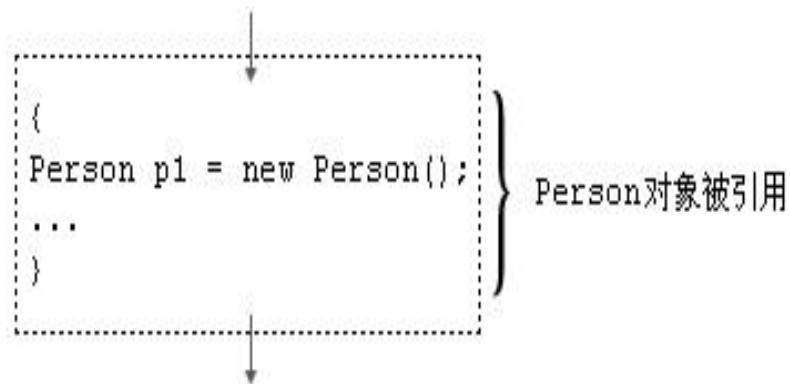
- Java虚拟机后台线程负责内存的回收。
- 垃圾强制回收机制：Java系统提供了方法“**System.gc()**”和“**Runtime.gc()**”方法来强制立即回收垃圾(但系统并不保证会立即进行垃圾回收)。
- 判断一个存储单元是否是垃圾的依据是：**该存储单元所对应的对象是否仍被程序所用。**

- 判断一个对象是否仍为程序所用的依据是：**是否有引用指向该对象。**
- Java的垃圾收集器自动扫描对象的动态内存区，对所引用的对象加标记，然后把没有引用的对象作为垃圾收集起来并释放出去。
- Java虚拟机可以自动判断并收集到“垃圾”，但一般不会立即释放它们的存储空间。
- Java系统自己定义了一套垃圾回收算法，用来提高垃圾回收的效率。

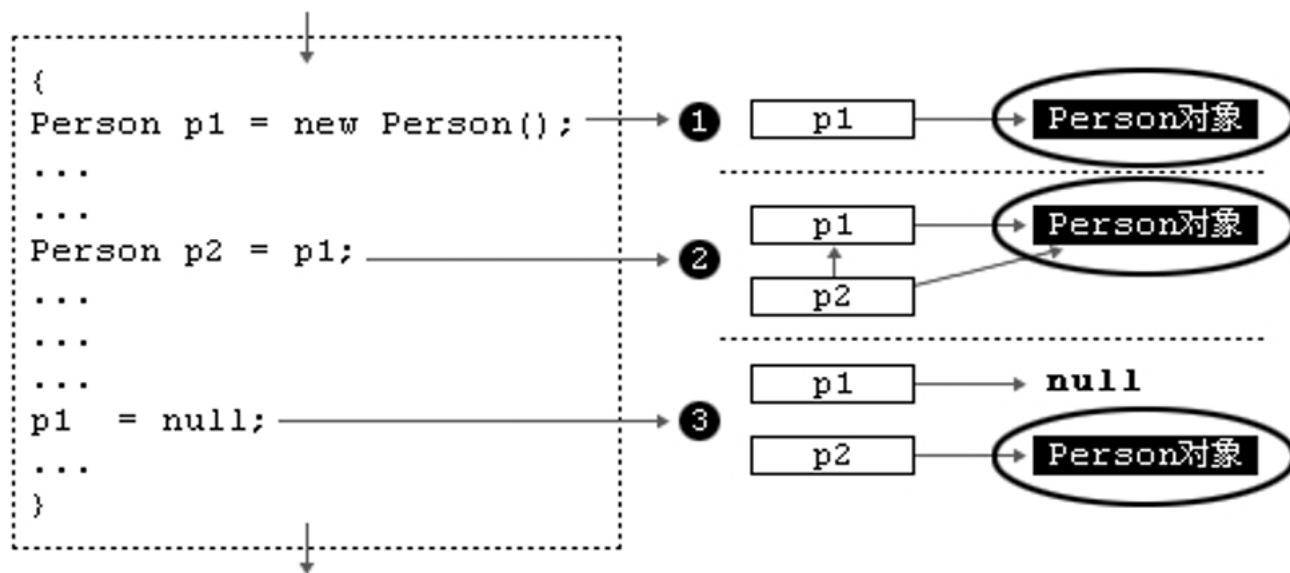
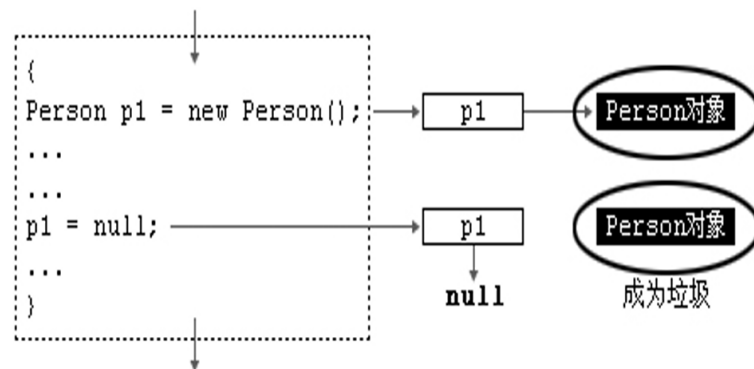
Java内存强制回收

- **System.gc();**强制系统回收垃圾内存
- **Runtime.gc();**强制系统回收垃圾内存
- Java没有提供析构方法，但提供了一个类似的方法：**protected void finalize()**。
- Java虚拟机在回收对象存储单元之前先调用该对象的finalize方法，如果该对象没有定义finalize方法，则java系统先调用该对象默认的finalize方法。

对象的生命周期



离开作用域p1失效, Person对象成为垃圾



J_Finalize. java

```
3 public class Student {
4
5     @Override
6     protected void finalize(){
7         System.out.println(x: "Finalize function is running!");
8     }
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85 public static void main(String[] args){
86     Student stu = new Student(studentId: "S20371324". studentName: "Elsa".
87         studentAge: 19, idNumber: "406
88         new Address(street: "Xueyuan
89         Building", unitNo: 501));
90     // int age = stu.getAge();
91     // Student.PhoneNumber book = stu.new PhoneNumber("15554673672");
92     System.out.println(stu);
93     new Student(studentId: "S20371325", studentName: "Tom",
94         studentAge: 19, idNumber: "406
95         new Address(street: "Xue
96         Building", unitNo: 501));
97     stu = null;
98     System.gc();
99 }
```

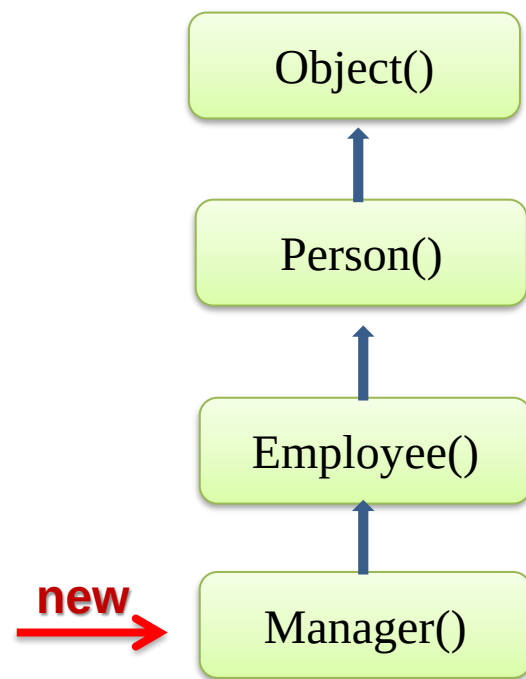
Student Tom is destroyed immediately

Student Elsa is destroyed

Invoke twice

小结 (1)

- **对象**是程序所处理数据的最主要的载体，数据以实例变量的形式存放在对象中。**每个对象在生命周期的开始阶段，Java虚拟机都需要为它分配内存，然后对它的实例变量进行初始化。**



小结 (2)

- **合理地使用内存**：每个对象都会占用一定的内存，而内存是有限的资源，为了合理地利用内存，在决定对象的生命周期时，应该遵循以下原则：
 - 当程序不需要再使用一个对象，应该及时清除对这个对象的引用，使它的内存可以被回收。
 - **重用已经存在的对象**。程序可利用已经存在的对象，而不是通过new语句来创建新的对象。
(Java反射)
- **匿名对象**：当一个对象不被程序的任何引用变量引用，对象就变成无用对象，它占用的内存就可以被垃圾回收器回收（**但不一定立即回收**）。

匿名对象概念和使用

- 我们也可以不定义对象的句柄，而直接调用这个对象的方法。这样的对象叫做匿名对象
- 例如：
 - `Student stu = new Student();` //stu是对象，名字是stu
 - `new Student();` //这个也是一个对象，但是没有名字，称为匿名对象
 - `new Student().show();` 匿名对象方法调用
- 匿名对象用完了之后就变成垃圾了，因为这个对象没有被栈内存中的变量指向，所有会被gc回收

匿名对象概念和使用

- 如果对一个对象只需要进行一次方法调用，那么就可以使用匿名对象。
- 我们经常将匿名对象作为实参传递给一个函数调用

```
teacher1.addStudent(new Student(studentId: "S20371324",  
studentName: "Elsa", studentAge: 19, idNumber: "400401....", addr));
```

匿名对象

```
39 | public void addStudent(Student stu) {  
40 |     students.add(stu);  
41 | }
```


主要内容

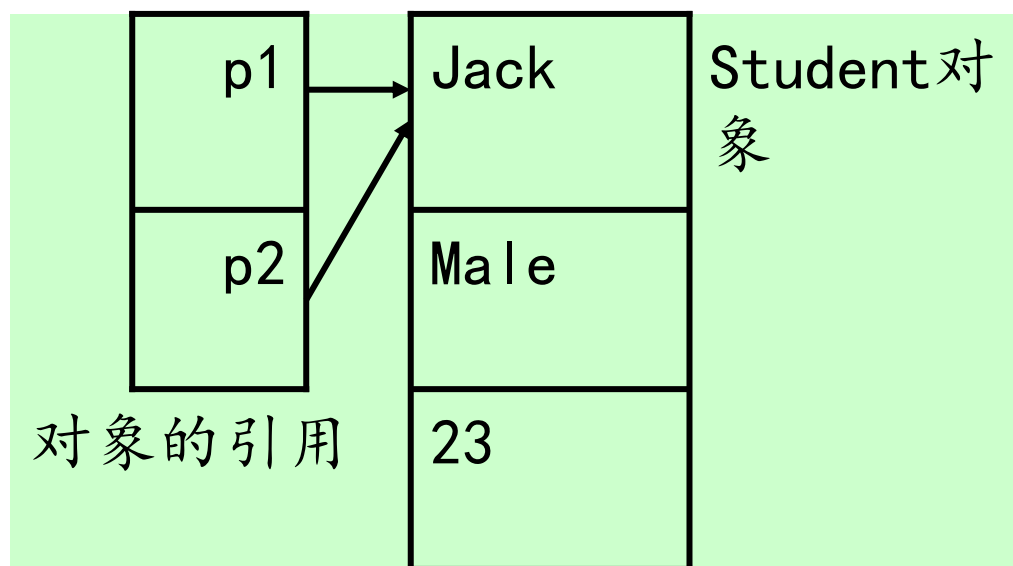
- 类的设计原则
- Java程序的基本结构
- 类的定义
- 进一步理解对象
 - this关键字
 - 对象的生成、使用和清除
 - 进一步理解引用类型变量
 - 对象序列化
- 类的静态属性和静态方法
- 进一步理解封装
- 单例模式

进一步理解引用类型变量

- 对象由两部分组成：
 - 对象的实体
 - 对象的引用

进一步理解引用类型变量

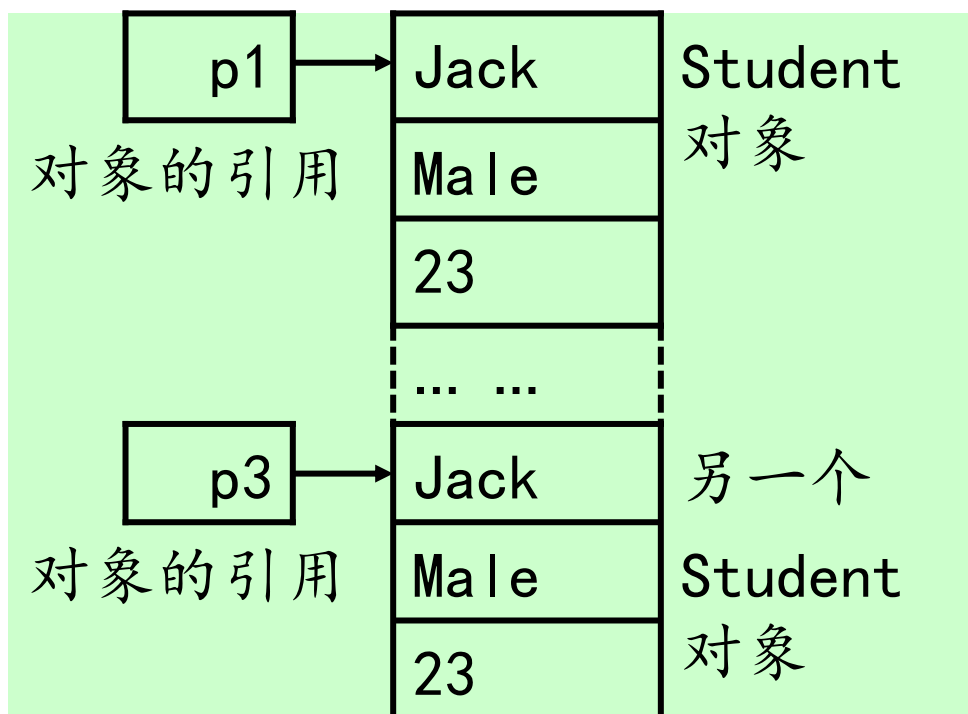
- Person p1=new Student (“Jack” , ” Male” , 23) ;
- Person p2=p1;



p1和p2指向同一个对象

进一步理解引用类型变量

- Student p1=new Student(“Jack”,”Male”,23);
- Student p3=new Student(“Jack”,”Male”,23);



P1和P3指向不同的对象

进一步理解引用类型变量

```
public Bank() {  
    customers = new Customer[5];  
}
```

```
public void addCustomer(String f, String l){  
    Customer customer = new Customer(f, l);  
    customers[numberOfCustomers] = customer;  
    numberOfCustomers++;  
}
```

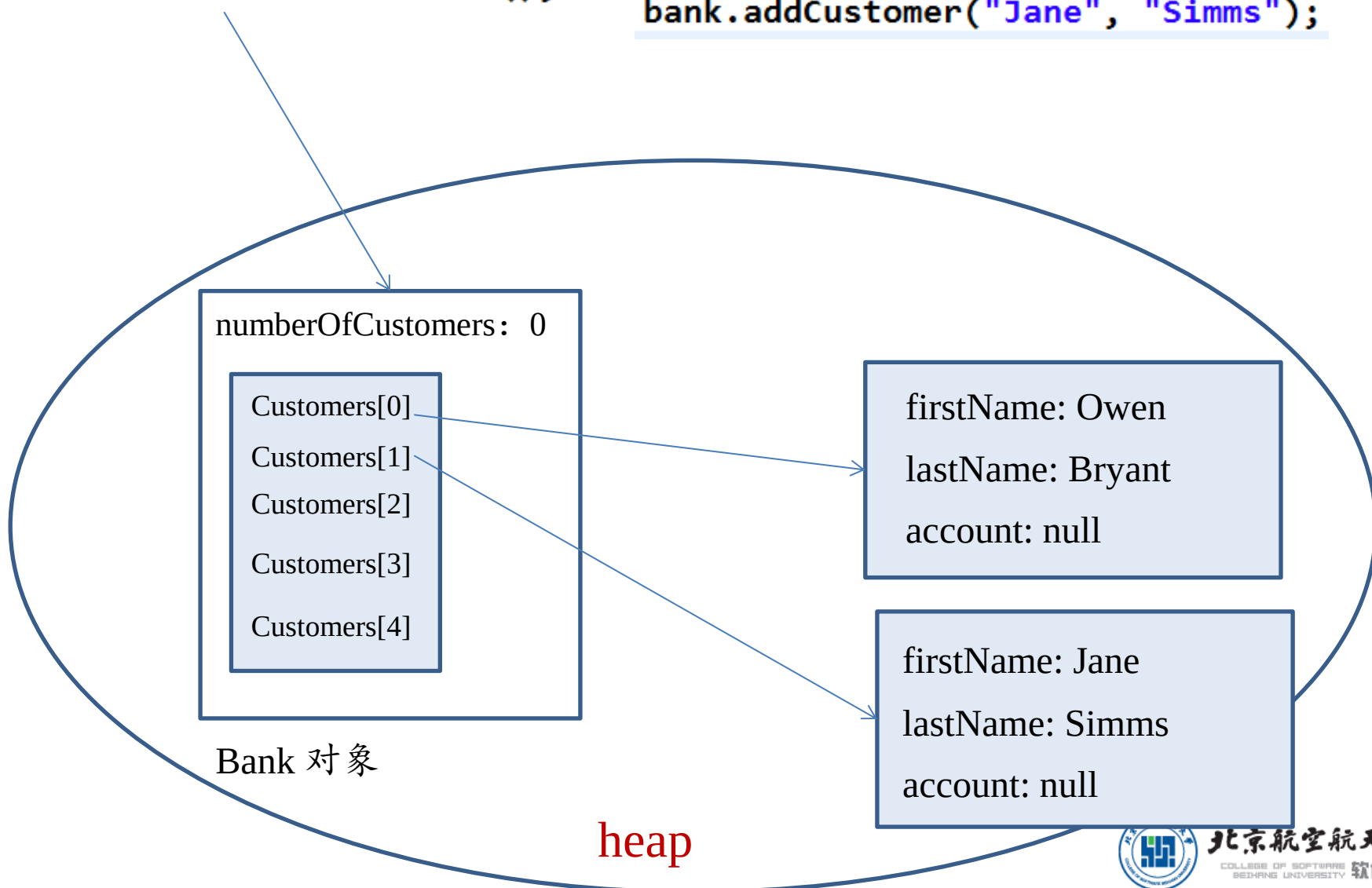
进一步理解引用类型变量

```
bank.addCustomer("Owen", "Bryant");
```

```
bank.addCustomer("Jane", "Simms");
```

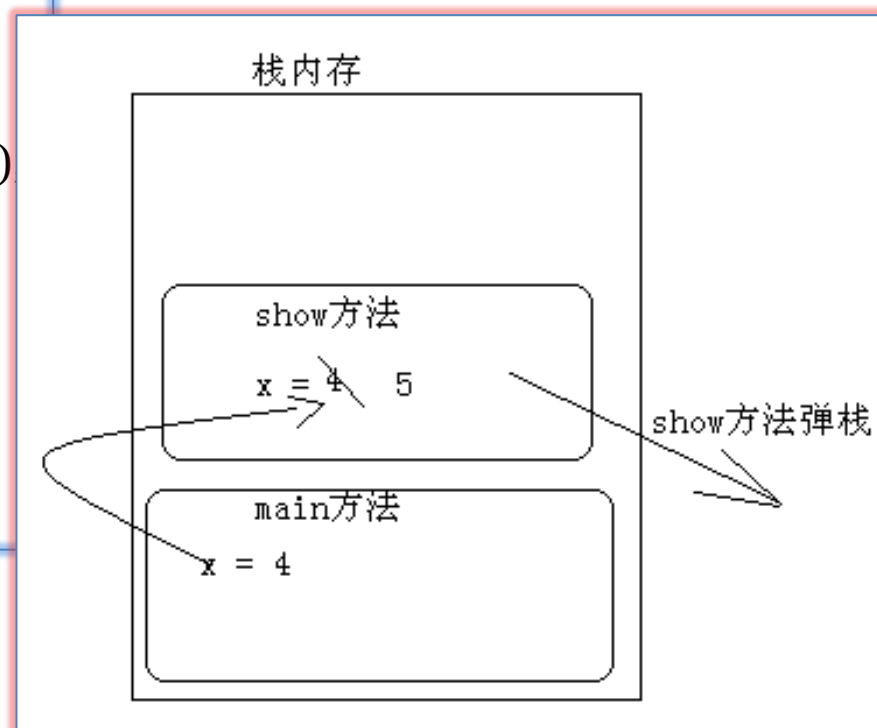
Bank

```
bank = new Bank();
```



基本类型和引用类型作为参数传递

```
class Demo
{
    public static void main(String[] args)
    {
        int x = 4;
        show(x);
        System.out.println("x="+x)
    }
    public static void show(int x)
    {
        x = 5;
    }
}
```



基本类型作为参数传递 (深拷贝)

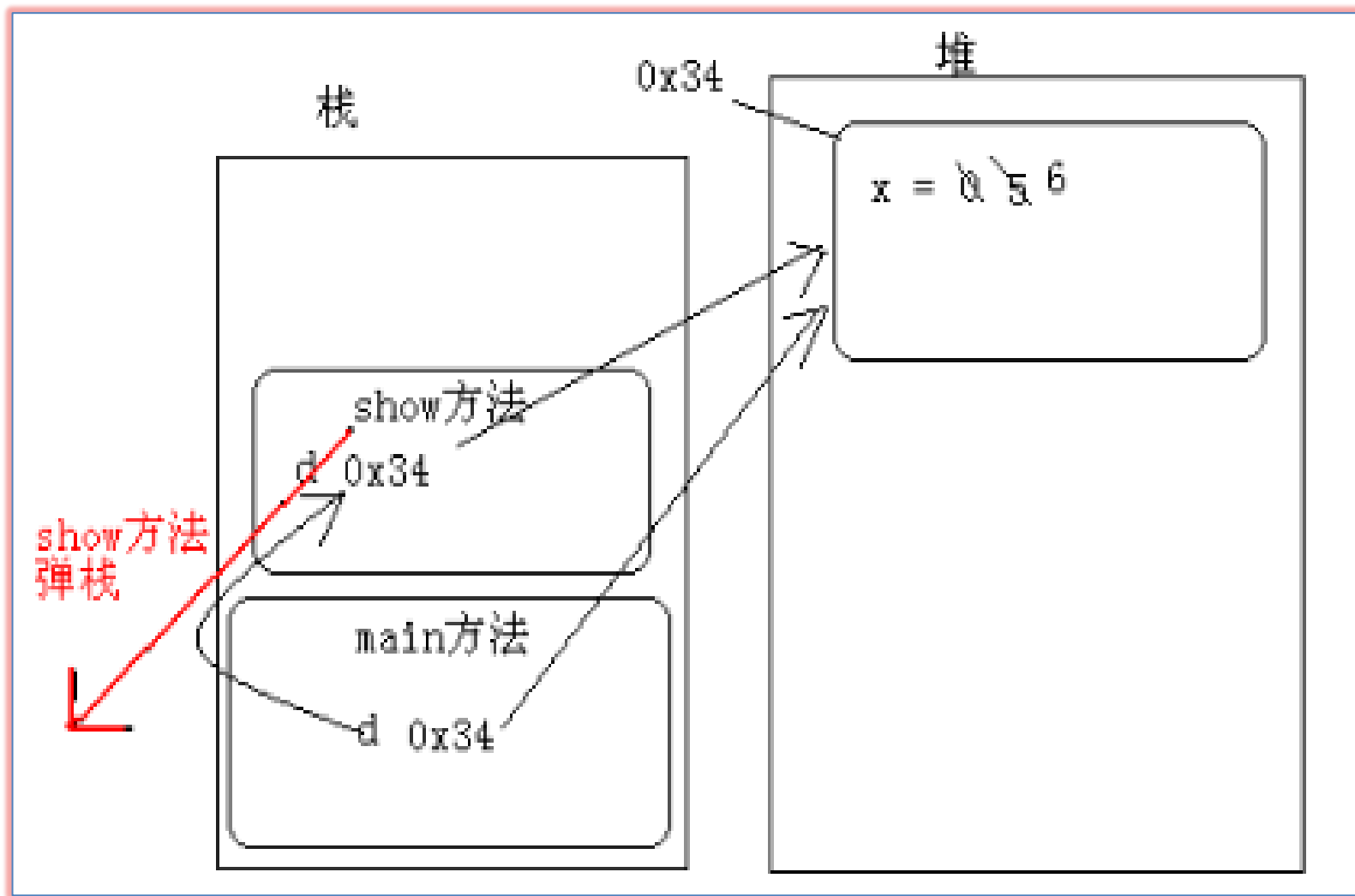
- 基本类型作为参数传递时，其实就是将基本类型变量x空间中的值复制了一份传递给调用的方法show()，当在show()方法中x接受到了复制的值，再在show()方法中对x变量进行操作，这时只会影响到show中的x。当show方法执行完成，弹栈后，程序又回到main方法执行，main方法中的x值还是原来的值。

当引用变量作为参数传递时 （浅拷贝）

```
class Demo
{
    int x ;
    public static void main(String[] args)
    {

        Demo d = new Demo();
        d.x = 5;
        show(d);
        System.out.println("x="+d.x);
    }
    public static void show(Demo d)
    {
        d.x = 6;
    }
}
```





当引用变量作为参数传递时

- 当引用变量作为参数传递时，这时其实是将引用变量空间中的内存地址(引用)复制了一份传递给了show方法的d引用变量。这时会有两个引用同时指向堆中的同一个对象。当执行show方法中的d.x=6时，会根据d所持有的引用找到堆中的对象，并将其x属性的值改为6.。
- 由于是两个引用指向同一个对象，不管是哪一个引用改变了引用的所指向的对象的中的值，其他引用再次使用都是改变后的值。

Person.java (分清楚全局变量和局部变量)

```
package com.buaa.classEx;
import java.io.*;
public class Person {
    private String name;
    private String sex;
    private int age;
    public Person() {
        int i=0;
        System.out.println("父类构造函数");
    }
    public Person(String n, String s, int a) {
        name = n;
        sex = s;
        age = a;
    }
    public void display() {
        System.out.println("姓名: " + name + ", " + "性别: " + sex + ", " + "年龄: " + age);
    }
}
```

全局变量 (堆)

局部变量 (栈)

Person.java

```
public static void main(String[] args) {  
    Person p0=new Person();///  
    p0.display();///  
    Person p = new Person("张三", "男", 20);  
    p.display();  
    int i=0;///  
    System.out.println(i);  
    System.out.println(p0.toString());///  
}  
  
}
```

- 自动初始化只用于成员变量，方法体中的局部变量不能被自动初始化，必须赋值后才能使用。

主要内容

- 类的设计原则
- Java程序的基本结构
- 类的定义
- 进一步理解对象
 - this关键字
 - 对象的生成、使用和清除
 - 进一步理解引用类型变量
 - 对象序列化
- 类的静态属性和静态方法
- 进一步理解封装
- 单例模式

- **存储对象到文件？**
- Java 提供了一种对象序列化的机制，该机制中，一个对象可以被表示为一个字节序列，该字节序列包括该对象的数据、有关对象的类型的信息和存储在对象中数据的类型。将序列化对象写入文件之后，可以从文件中读取出来，并且对它进行反序列化，也就是说，对象的类型信息、对象的数据，还有对象中的数据类型可以用来在内存中新建对象。

对象的序列化

Run | Debug

```
7 public static void main(String [] args)
8 {
9     Employee e = new Employee(name: "Reyan Ali", address: "Phokka Kuan,
    Ambehta Peer", number: 103);
10 try
11 {
12     FileOutputStream fileOut =
13         new FileOutputStream(name: "/tmp/employee.ser");
14     ObjectOutputStream out = new ObjectOutputStream(fileOut);
15     out.writeObject(e);
16     out.close();
17     fileOut.close();
18     System.out.println("Serialized data is saved in /tmp/employee.ser");
19 } catch (IOException i)
20 {
21     i.printStackTrace();
22 }
```


对象的序列化

```
6      public static void main(String[] args) {
7          Employee e = null;
8          try {
9              FileInputStream fileIn = new FileInputStream(name: "/tmp/employee.
10                 ser");
11              ObjectInputStream in = new ObjectInputStream(fileIn);
12              e = (Employee) in.readObject();
13              in.close();
14              fileIn.close();
15          } catch (IOException i) {
16              return;
17          } catch (ClassNotFoundException c) {
18              return;
19          }
20          System.out.println(x: "Deserialized Employee...");
21          System.out.println("Name: " + e.name);
22          System.out.println("Address: " + e.address);
23          System.out.println("Number: " + e.getNumber());
```

主要内容

- 类的设计原则
- Java程序的基本结构
- 类的定义
- 进一步理解对象
- 类的静态属性和静态方法
 - 静态属性
 - 静态方法
- 进一步理解封装
- 单例模式

- 问题一：某些数据或者函数不要和class object绑在一起。
- 问题二：不论产生多少个对象，或不存在任何对象的情况下，某些特定数据的存储空间都只有一份；
 - 例如：统计出从Person共new出多少个对象？

- 通过**关键字static**，便可以处理这两种情况。
- Java中把这种**代表类范围信息**的变量用关键字static修饰。
- **类变量和类方法**：当你将某个数据成员或某个成员函数声明为static时，它就不再局限于所属的class object上。

Static的使用场合

- 一. 用static修饰的属性（变量）称为静态属性，又叫类变量；
- 二. 用static修饰的方法称为静态方法，又叫类方法（静态方法里, 无this）；

类的静态属性访问方法

- 即使没有创建该类的具体对象，类中的 `static` 类成员也会存在，这时可以通过：
 1. 类名. 静态变量
 2. 类名. 静态方法
- 当你将某个数据成员或某个成员函数声明为 `static` 时，它就不再局限于所属的 `class` `object` 上

案例(统计出从Person共new出多少个对象?)

```
3  public class People{
4      String name;
5      String sex;
6      int age;
7      public static int count;
8      public int _count;
9
10     public People(String name, String sex, int age){
11         this.name = name;
12         this.age = age;
13         this.sex = sex;
14         count ++;
15         _count ++;
16     }
17
18     public String toString(){
19         String ret = "name: " + name + " sex: " + sex + " age: " + age;
20         return ret;
21     }
22 }
```

案例(统计出从Person共new出多少个对象?)

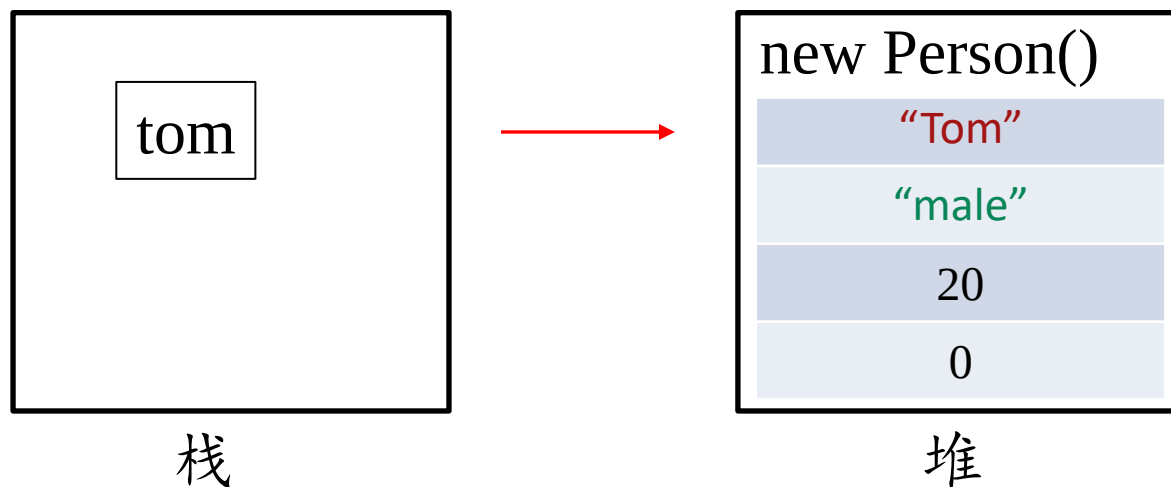
```
class PeopleTest{
```

Run | Debug

```
public static void main(String[] args){  
    System.out.println("initial count: " + People.count);  
    System.out.println(x: "===== Tom =====");  
    People p1 = new People(name: "Tom", sex: "male", age: 20);  
    System.out.println("_count: " + p1._count);  
    System.out.println("count: " + p1.count);  
    System.out.println("count: " + People.count);  
  
    System.out.println(x: "===== Mary =====");  
    People p2 = new People(name: "Mary", sex: "female", age: 20);  
    System.out.println("_count: " + p2._count);  
    System.out.println("count: " + p2.count);  
    System.out.println("count: " + People.count);  
}
```


程序运行时的内存状态

```
Person tom = new Person("Tom", "male", "20");
```



变量声明不为对象分配内存空间，只分配引用空间；对象的引用类似于指针，是32/64位地址空间，它的值指向堆中的数据结构。

```
public class People{
```

```
    String name;
```

```
    String sex;
```

```
    int age;
```

```
    public static int count; //类变量
```

```
    public int _count; //成员变量
```

可以存放在栈或堆吗？

```
    public People(String name, String sex, int age){
```

2024/9/26

```
        this.name = name;
```

Xiang Gao

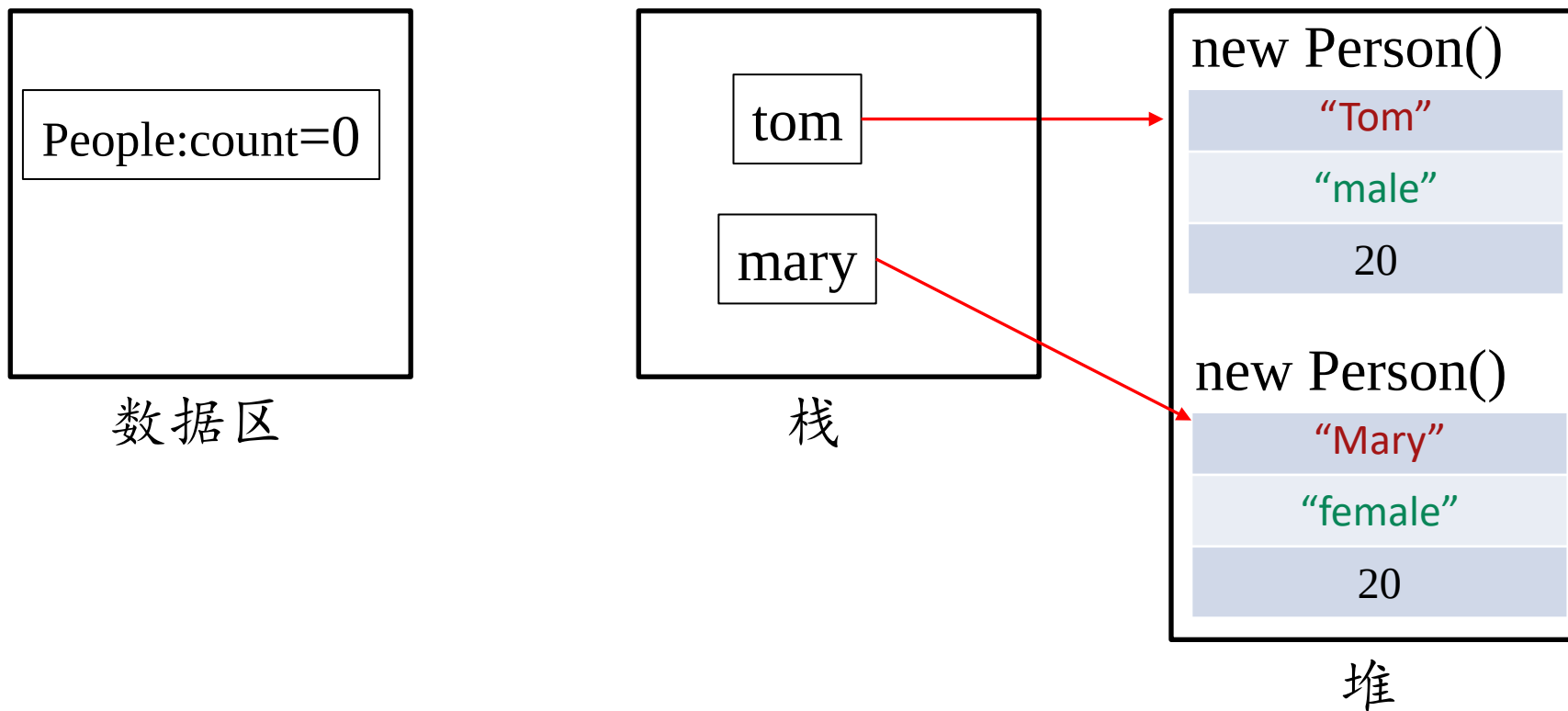
程序运行时的内存占用

代码区 (code area)	存放程序的代码部分
数据区 (data area)	存放程序的全局数据和静态数据
堆区 (heap area)	存放程序动态申请的数据
栈区 (stack area)	存放程序的局部数据和参数

程序运行时的内存占用

```
Person tom = new Person("Tom", "male", "20");
```

```
Person mary = new Person("Mary", "female", "20");
```



Student.java

```
class Student {  
    static int num=10; //类变量，保存在全局数据区，所有的对象共享  
    int number; //实例变量，保存在heap，每个对象有一份独立的拷贝  
    int add1() { //add1()方法退出时，内存中的值可以保留  
        num = num + 1;  
        return num;  
    }  
    int add2() { //add2()方法退出时，内存中的值不能保留  
        number = number + 1;  
        return number;  
    }  
}
```

StuDemo.java

```
class Student {  
    static int num=10;    //类变量，保存在全局数据区，所有的对象共享  
    int number;          //实例变量，保存在heap，每个对象有一份独立的拷贝  
    int add1() { //add1()方法退出时，内存中的值可以保留  
        num = num + 1;  
        return num;  
    }  
    int add2() { //add2()方法退出时，内存中的值不能保留  
        number = number + 1;  
        return number;  
    }  
}
```

```
10  
zhang.num = 11  
zhang.number = 1  
wang.num = 12  
wang.number = 1  
jiang.num = 13  
jiang.number = 1
```

```
public static void main(String args[ ]){  
    System.out.println(Student.num);  
    Student zhang=new Student( );//  
    Student wang=new Student( );  
    Student jiang=new Student( );  
    System.out.println("zhang.num="+zhang.add1( ));//1  
    System.out.println("zhang.number="+zhang.add2( ));//1  
    System.out.println("wang.num="+wang.add1( ));//2  
    System.out.println("wang.number="+wang.add2( ));//1  
    System.out.println("jiang.num="+jiang.add1( ));//3  
    System.out.println("jiang.number="+jiang.add2( ));//  
}
```

主要内容

- 类的设计原则
- Java程序的基本结构
- 类的定义
- 进一步理解对象
- 类的静态属性和静态方法
 - 静态属性
 - 静态方法
- 进一步理解封装
- 单例模式

静态方法与非静态方法的区别

- 两者本质上的区别是：静态方法是在类中使用static修饰的方法，可以由类名直接调用。而非静态方法是不加static关键字的方法，只有在类被实例化成对象时，对象才能调用该方法。

静态方法

```
3 public class People{
4     String name;
5     String sex;
6     int age;
7     public static int count; //类变量
8     public int _count;      //成员变量
9
10    public People(String name, String sex, int age){
11        this.name = name;
12        this.age = age;
13        this.sex = sex;
14        count ++;
15        _count ++;
16    }
17
18    public static int getCount(){ //静态方法
19        return count;
20    }
```

思考静态方法能否访问普通成员变量，为什么？

静态方法与非静态方法的区别

- 两者本质上的区别是：静态方法是在类中使用static修饰的方法，可以由类名直接调用。而非静态方法是不加static关键字的方法，只有在类被实例化成对象时，对象才能调用该方法。
- 静态方法中只能直接调用静态成员或者方法，不能调用非静态方法或者非静态成员，而非静态方法既可以调用静态成员或者方法又可以调用其他的非静态成员或者方法。

静态方法不能直接引用非静态方法

```
3 public class Test {  
4     public int sum(int a, int b){  
5         return a+b;  
6     }  
7  
8     Run | Debug  
9     public static void main(String []args){  
10         int res = sum(a: 1, b: 2);  
11         System.out.println("the sum is " + res);  
12     }  
}
```

Compilation Error: Cannot make a static reference to the non-static method sum(int, int) from the type Test

- 静态方法只能访问静态方法和静态成员
- 非静态方法要被实例化才能被静态方法调用

静态方法只能直接访问静态方法和静态成员

```
3 public class Test {  
4     public static int sum(int a, int b){ //加入static关键字, 变成静态方法  
5         return a+b;  
6     }  
7  
8     Run | Debug  
9     public static void main(String []args){  
10         int res = sum(a: 1, b: 2);  
11         System.out.println("the sum is " + res);  
12     }  
}
```

非静态方法要被实例化才能被静态方法调用

```
3 public class Test {  
4     public int sum(int a, int b){  
5         return a+b;  
6     }  
7  
8  
9     Run | Debug  
10    public static void main(String []args){  
11        Test test = new Test();  
12        int res = test.sum(a: 1, b: 2);  
13        System.out.println("the sum is " + res);  
14    }  
15 }
```

- 一. 你知道`main()`为什么要用`static`修饰的原因了吗?
- 二. `main()`方法中的`static`可以去掉吗?

类的静态属性和静态方法(小结)

- 类变量和类方法

1. 类的变量和方法是由关键字 static 修饰的。

2. 类变量和类方法由该类产生的所有实例共享。

3. 调用方式:

- 类名\对象名. 静态变量
- 类名\对象名. 静态方法

- 实例变量和方法

1. 每一个实例拥有一份独立拷贝，因此每个实例对象的数据是独立且唯一的。

2. 调用方式:

1. 对象名. 事例变量
2. 对象名. 事例方法

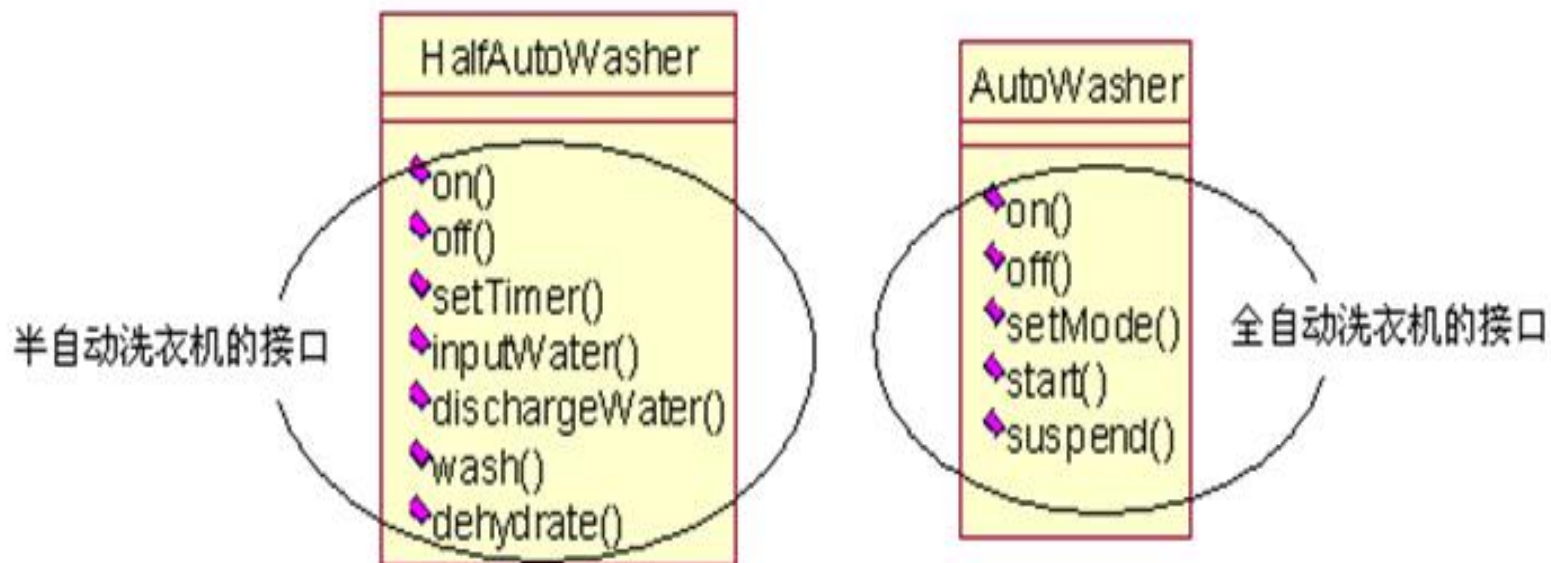
类的静态属性和静态方法（小结）

- 即使没有创建该类的具体对象，类中的static类成员也会存在，这时可以通过：
 1. 类名.静态变量
 2. 类名.静态方法

主要内容

- 类的设计原则
- Java程序的基本结构
- 类的定义
- 进一步理解对象
- 类的静态属性和静态方法
- 进一步理解封装
 - 信息隐藏的必要性
- 单例模式

深刻理解封装（从系统的角度）



半自动洗衣机 HalfAutoWasher

全自动洗衣机 AutoWasher

用半自动洗衣机洗衣服（界面欠清晰，不容易操作）

```
HalfAutoWasher washer=new  
HafAutoWasher();
```

```
//开始洗衣服  
washer.on(); //开机
```

```
//洗涤  
washer.inputWater(); //放水  
washer.setTimer(5); //定时5分钟  
washer.wash(); //洗涤  
washer.dischargeWater(); //排水
```

```
//第一次清洗  
washer.inputWater(); //放水  
washer.setTimer(5); //定时5分钟  
washer.wash(); //洗涤  
washer.dischargeWater(); //排水
```

```
//第二次清洗
```

```
washer.inputWater(); //放水  
washer.setTimer(5); //定时5分钟  
washer.wash(); //洗涤  
washer.dischargeWater(); //排水
```

```
//为衣服脱水
```

```
washer.setTimer(8); //定时8分钟  
washer.dehydrate(); //脱水
```

```
washer.off(); //关机
```



用全自动洗衣机洗衣服（界面清晰，容易操作，调用更简单）

```
AutoWasher washer=new AutoWasher();
```

```
//开始洗衣服
```

```
washer.on(); //开机
```

```
washer.setMode("标准模式"); //设置洗衣机模式
```

```
//开始洗衣服，洗衣结束后，30分钟内洗衣机会自动关机
```

```
washer.start();
```



深刻理解封装（从系统的角度）

- 把尽可能多的东西藏起来，对外提供简洁的接口
- 系统的封装程度越高，那么它的相对独立性就越高，而且使用起来更方便。



深刻理解封装（从系统的角度）

- 封装是指隐藏对象的属性和实现细节，仅仅对外公开接口。
- 封装的优点：
 - （1）便于使用者正确的方便地理解和使用系统，防止使用者错误修改系统的属性。
 - （2）有助于建立各个系统之间的松耦合关系，提高系统的独立性。当某一个系统的实现发生变化，只要它的接口不变，就不会影响到其他的系统。
 - （3）提高软件的可重用性，每个系统都是一个相对独立的整体，可以在多种环境中得到重用。
 - （4）降低了构建大型系统的风险，即使整个系统不成功，个别的独立子系统有可能依然是有价值的。



主要内容

- 类的设计原则
- Java程序的基本结构
- 类的定义
- 进一步理解对象
- 类的静态属性和静态方法
- 进一步理解封装
 - 信息隐藏的必要性
- 单例模式

封装的含义：现实世界案例（机箱）：

- 一台电脑，它是由CPU、主板、显卡、内存、硬盘、电源等部件组成，我们将这些部件组装在一起就可以使用电脑了。
- 如果这些部件散落在外面，很容易造成不安全因素，于是，使用机箱壳子，把这些部件都装在里面。
- 同时，会在机箱壳上留下一些插口等，若不留插口，大家想想会是什么情况？
- 总结：机箱其实就是隐藏了设备的细节，对外提供了插口以及开关等访问内部细节的方式。



为什么要使用封装



- 下面代码有什么问题？

```
Dog d = new Dog();  
d.age = 1000;
```



不合理的赋值

- 如何解决上面设计的缺陷？

使用**封装**



封装 (encapsulation)

- 封装是面向对象方法的核心思想之一，他有两个含义
 - 一层含义是把对象的属性和行为看成为一个密不可分整体，将这两者封装在一个不可分割的独立单位（即对象）中。
 - 另一层含义指信息隐藏，把不需要让外界知道的信息隐藏起来，有些对象的属性及行为允许外界用户知道或使用，但不允许更改，而另一些属性和行为则不允许外界知晓或只允许使用对象的功能，而尽可能隐藏对象的功能实现细节。



封装的示例(访问权限加以控制)



花园：所有人可以访问

田地：本人及儿子可以访问

房屋：本人及同一家族可以访问

小金库：归自己私有

为了安全，我们对花园、天地、房屋及小金库加入了访问权限



封装的示例

实体法拉力公司

属性

电话号码
电子邮箱
产品种类
现有存货数量
雇员详情
所需物料详情

方法

接收订单
备件的制造方法
计算备件的制造成本
计算利润率

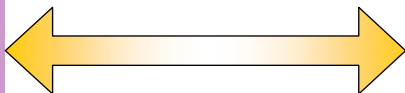
实体迪斯尼汽车公司

属性

电话号码
电子邮箱
汽车种类
汽车规格
雇员详情
库存详情
经销商详情

方法

发出订单
汽车的组装方法
计算汽车的组装成本
计算工资
编制必要报告



信息隐藏的必要性 (Information Hiding)

MyDate
+day: int +month: int +year: int

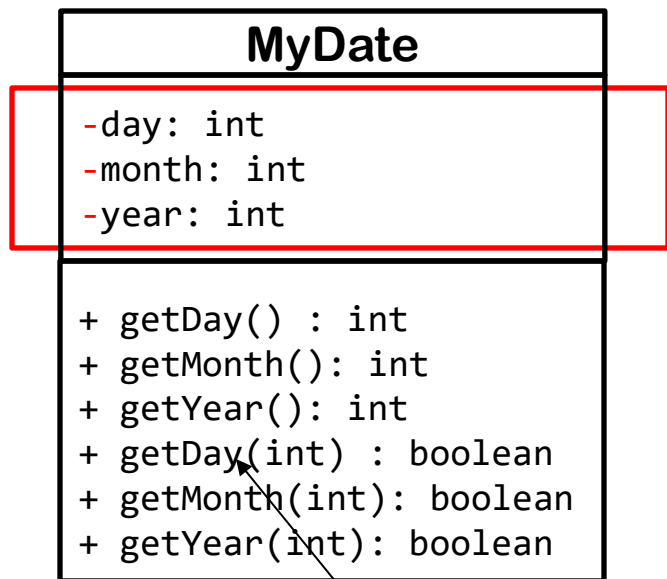
```
d.day = 32;  
// invalid day
```

```
d.Month = 2; d.day = 30;  
// plausible but wrong
```

```
d.day = d.day + 1;  
// no check for wrap around
```



信息隐藏的必要性 (Information Hiding) — 解决方案



Verify whether a day is valid

```
MyData d = new MyData();
```

```
d.setDay(32);  
// invalid day, return  
false
```

```
d.setMonth(2);  
d.setDay(30);  
// plausible but wrong  
// setDay returns false
```

```
d.setDay(d.getDay() + 1);  
// return false if wrap  
around needs to occur
```



信息隐藏的必要性 (Information Hiding) — 更好的方案

MyDate	
- day: int - month: int - year: int	
+ getDay() : int + getMonth(): int + getYear(): int + getDay(int) : boolean + getMonth(int): boolean + getYear(int): boolean - isValid(int): boolean	



案例分析（1）Employee 类

```
class Employee {  
    String name;  
    String id;  
    String gender;  
  
    public void work() {  
        System.out.println(id + ":" + name + ":" + gender + " 努力工作中!!!");  
    }  
}
```



测试类（假设两个类在同一个包中）

```
public class EmployeeDemo {  
    public static void main(String[] args) {  
        // 创建对象  
        Employee jack = new Employee();  
  
        // 进制通过类名.成员的形式调用成员。初始化实例变量  
        jack.name = "jack";  
        jack.id = "123456";  
        jack.gender = "男";  
  
        // 调用成员方法  
        jack.work();  
        System.out.println();  
  
        // 传入非法的参数  
        jack.gender = "不是男人";  
        jack.work();  
    }  
}
```



代码分析：

- 缺陷：如果不使用封装，很容易赋值错误，并且任何人都可以更改，造成信息的不安全。
- 问题解决：使用封装



Employee 类（更好的解决方案）

```
class Employee {  
    //使用了private修饰了成员变量  
    private String name;  
    private String id;  
    private String gender;  
  
    public void work() {  
        System.out.println(id + ":" + name + ":" + gender + " 努力工作中!!!");  
    }  
}
```



为什么编译出错？

```
public class EmployeeDemo {  
    public static void main(String[] args) {  
        // 创建对象  
        Employee jack = new Employee();  
  
        //编译报错  
        jack.name = "jack";  
        jack.id = "123456";  
        jack.gender = "男";  
  
        // 编译报错  
        jack.gender = "不是男人";  
        jack.work();  
    }  
}
```



更好的解决方案

- 将所有的成员变量封装加上private，对外提供公开的用于设置对象属性的public方法
 - 设置set
 - 获取get
- Employee类的gender的修饰符修改为private后，无法在类外调用，那么如何给gender设置合适的值？
 - 在setGender()方法中加入逻辑判断，过滤掉非法数据。



```
class Employee {  
    private String name;  
    private String id;  
    private String gender;  
    // 提供公有的get set方法  
    public String getName() {  
        return name;  
    }  
}
```

哈哈，这样，别人就不能恶搞对象的性别了

```
    public void setGender(String gen) {  
        if ("男".equals(gen) || "女".equals(gen)) {  
            gender = gen;  
        } else {  
            System.out.println("请输入\"男\"或者\"女\"");  
        }  
    }  
  
    public void work() {  
        System.out.println(id + ":" + name + ":" + gender + "努力工作中!!!");  
    }  
}
```



案例分析（2）：该看的看，不该看的别看

属性私有
(用private关键字修饰)

```
public class Student {  
    private String name; // 姓名  
    private int rp;      // 人品 (取值在1-10之间, 越高越好)
```

```
    public int getRp() {  
        return rp;  
    }
```

公用的访问方法, 私有设置方法

```
    private void setRp(int rp) {  
        if (rp < 1 || rp > 10) {  
            System.out.println("错误! 人品值应该在1-10之间!");  
            this.rp = 1; // 人品不符合要求, 赋予默认值1  
        } else {  
            this.rp = rp;  
        }  
    }
```

.....

```
}
```



案例分析（2）：该看的看，不该看的别看

哈哈，这样，别人就不能恶搞对象的人品了

```
public class Student2Test {  
    public static void main(String[] args) {  
        //实例化学员对象,对其属性进行初始化  
        Student2 xiaoxin = new Student2();  
        xiaoxin.setName("小新");  
        xiaoxin.setRp(-1);  
        //小新自我介绍  
        xiaoxin.introduction();  
    }  
}
```

编译错误

➤ 我们可以通过公有的getter（取值）、setter（赋值）方法访问这两个属性，而且在人品的赋值方法中加入了属性访问的限制，成功的实现了对小明人品的拯救。



小结:

- 通过封装：
 - 1: 隐藏了类的具体实现
 - 2: 操作简单
 - 3: 提高对象数据的安全性
 - 4: 减少了冗余代码，数据校验等写在方法里，可以复用
- 所以在解决实际问题的時候，我們需要對成員變量和成員方法進行歸類，設置訪問級別，提供不同程度的對外訪問權限。



主要内容

- 类的设计原则
- Java程序的基本结构
- 类的定义
- 进一步理解对象
- 类的静态属性和静态方法
- 进一步理解封装
 - 信息隐藏的必要性
- 单例模式

案例分析（1）：

- 问题：构造方法可以使用`private`修饰吗？
- 问题：希望程序使用者不能随意自己创建对象，只能获取一个单独的对象，即一个类在内存中只有一个实例（对象）存在，如何实现？
 - 例如：比如多程序访问同一个配置文件，希望多程序操作的都是同一个配置文件中的数据，那么就需要保证该配置文件对象的唯一性。
 - 又比如系统日志，多线程访问一个程序，需要记录日志，需要把日志单例化。你不希望看到N多个日志文件实例吧？



单例模式

- 单例模式：
 - 一个类在内存中只有一个实例（对象）存在，该类一般没有属性。
 - 无法继承，所以无法扩展，无法更改它的实现。
- 重点：
 - 1、单实例模式及其适用场合
 - 2、加深对private的理解
 - 3、加深对static的理解



“饿汉式”单实例模式

```
public class Singleton
{
    //静态的。保留自身的引用。类加载时就初始化
    private static Singleton test = new Singleton();
    //必须是私有的构造函数
    private Singleton(){}
    //公共的静态的方法。
    public static Singleton getInstance() {
        return test;
    }
}
```



“饿汉式”单实例模式

- 是否 Lazy 初始化：否
- 是否多线程安全：是
- 描述：这种方式比较常用，但容易产生垃圾对象。
- 优点：没有加锁，执行效率会提高。
- 缺点：类加载时就初始化，浪费内存。
- 特点：它基于 classloader 机制避免了多线程的同步问题，不过，instance 在类装载时就实例化



“懒汉式”单实例模式

```
public class Singleton  
{
```

懒汉式：单例的延迟加载方式。
延迟：需要的时候，再创建

//静态的。保留自身的引用。

```
private static Singleton test = null;
```

//必须是私有的构造函数

```
private Singleton(){}  
//公共的静态的方法。
```

```
public static Singleton getInstance() {  
    if(test == null)
```

```
    {
```

```
    {
```

```
        test = new Singleton();
```

```
    }
```

```
    return test;
```

```
    }
```

```
}
```



“懒汉式”单实例模式

- 是否 Lazy 初始化：是
- 是否多线程安全：否
- 描述：这种方式是最基本的实现方式，这种实现最大的问题就是不支持多线程。因为没有加锁 synchronized，所以严格意义上它并不算单例模式。
- 特点这种方式 lazy loading 很明显，不要求线程安全，在多线程不能正常工作。



但是单例的延迟加载方式会出现多线程中的安全问题，需要对上面的代码进行修改（了解）

```
class Single2
{
    private static Single s = null;
    private Single(){}
    public static Single getInstance()
    {
        if(s==null)
        {
            synchronized(Single.class)
            {
                if(s==null)

                    s = new Single();

            }
        }
        return s;
    }
}
```



单实例模式及其适用场合

- 只需要使用一个单独的资源，并且需要共享这个单独资源的状态信息时，就能用到单例模式。
- 需要频繁的创建和销毁的对象；
- 创建对象时耗时过多或耗费资源过多，但又经常用到的对象；
- 工具类对象；
- 频繁访问数据库或文件的对象。



单实例模式优点

- 由于单例模式在内存中只有一个实例，减少了内存开销。
- 系统内存中该类只存在一个对象，节省了系统资源，对于一些需要频繁创建销毁的对象，使用单例模式可以提高系统性能。
- 单例模式可以在系统设置全局的访问点，优化和共享资源访问。
- 例如一个播放器程序，当用户打开一个播放音乐界面，再想打开另一个音乐播放时，之前的界面就关闭了。这就是一个单例模式的具体应用



单例模式：小结

- 单例类是指仅有一个实例的类。
- 在系统中具有唯一性的组件可作为单例类，**这种类的实例通常会占用较多的内存，或者实例的初始化过程比较冗长，因此随意创建这些类的实例会影响系统的性能。**



代码块 {...}



2024/9/26



北京航空航天大学
COLLEGE OF SOFTWARE
BEIHANG UNIVERSITY 软件学院

构造代码块

```
3 public class CodeBlocks {
4     private int number;
5     private String str;
6
7     {
8         System.out.println(x: "initializing number!");
9         number = 100;
10        str = "";
11    }
12
13    public CodeBlocks(){
14        System.out.println(x: "Constructor 1 of CodeBlocker is running -->");
15    }
16
17    public CodeBlocks(int num){
18        this.number = num;
19        System.out.println(x: "Constructor 2 of CodeBlocker is running -->");
20    }
21 }
```

构造代码块会在创建对象时被调用，每次创建时都会被调用，优先于类构造函数执行

静态构造代码块

```
3 public class CodeBlocks {
4     private int number;
5     private static int num;
6     private String str;
7
8     {
9         System.out.print(s: "code block -->");
10        number = 100;
11        str = "";
12    }
13
14    static {
15        System.out.print(s: "static code block -->");
16        num = 10;
17    }
18
19    public CodeBlocks(){
```

静态构造代码块只在类第一次加载时执行一次，之后不再执行，而非静态代码块在每new一次就执行一次

静态代码块与非静态代码块的异同点

- 静态代码块：用`static`来修饰初始化语句块，这样的语句块常称为静态初始化语句块
- 相同点：
 1. 都是在JVM加载类时且在构造方法执行之前执行，在类中都可以定义多个。
 2. 一般在代码块中对一些`static`变量进行赋值。

静态代码块与非静态代码块的异同点

- 不同点:

1. 静态代码块在非静态代码块之前执行:

- ① 静态代码块—>非静态代码块—>构造方法

2. 静态代码块只在第一次加载时执行一次，之后不再执行，而非静态代码块在每new一次就执行一次

CodeBlocks

== first new CodeBlocks ==

static code block -->

code block -->

constructor 1 of CodeBlocker -->

normal function

==second new CodeBlocks ==

code block -->

constructor 1 of CodeBlocker -->

normal function

```
3 public class CodeBlocks {
4     private int number;
5     private static int num;
6     private String str;
7
8     {
9         System.out.print(s: "code block --> ");
10        number = 100;
11        str = "";
12    }
13
14    static {
15        System.out.print(s: "static code block --> ");
16        num = 10;
17    }
18
19    public CodeBlocks(){
20        System.out.print(s: "Constructor 1 of CodeBlocker --> ");
21    }
22
23    public void test(){
24        System.out.println(x: "normal function");
25    }
26
27 }
```

```
System.out.println(x: "===== first new CodeBlocks =====");
```

```
CodeBlocks cb = new CodeBlocks();
```

```
cb.test();
```

```
System.out.println(x: "===== second new CodeBlocks =====");
```

```
CodeBlocks cb2 = new CodeBlocks();
```

```
cb2.test();
```



静态代码块

- 一. 静态代码块只能定义在类里面，它独立于任何方法，不能定义在方法里面。
- 二. 静态代码块里面声明的变量都是局部变量，只在本块内有效。
- 三. 静态代码块会在类被加载时自动执行，而无论加载者是JVM还是其他的类。
- 四. 一个类中允许定义多个静态代码块，执行的顺序根据定义的顺序进行。
- 五. 静态代码块只能访问类的静态成员，而不允许访问实例成员。



staticBlock. java

This is static block.
This is main method.
counter=5

```
public class staticBlock {  
  
    private static int counter;  
    public static void main(String args[]) {  
        System.out.println("This is main method.");  
        System.out.println("counter="+counter);  
    }  
  
    // 定义一个静态代码块,静态代码块的执行早于main()的执行  
    static {  
        System.out.println("This is static block.");  
        counter = 5; //初始化counter  
    }  
}
```

Take Away Message

- `static`方法中不能直接调用非`static`的域或方法（必须通过对象名引用）。
 1. `static`函数并不需要先产生任何对象，就可以通过类名来调用。
 2. `non-static`数据/函数和对象绑定（原因）。
 3. 在`static`函数中“直接”取用`non-static`数据/函数，会产生语法错误。